



Computer Games Development CW208

GDD

Year IV

[Jad Fuhr]

[C00290965]

[12/04/2026]

Acknowledgements

I would like to thank my lecturers, supervisor, and peers at SETU for their guidance throughout this project.

I also acknowledge the creators of SFML and the nlohmann JSON library, whose tools formed the foundation of this project.

Game Overview

PaleoPals is a 2D mining, exploration, and fossil-collection game where the player digs through layered terrain to uncover dinosaur fossils, amber, and other collectibles. These items can be displayed in a museum, sold for money, or used to unlock cosmetic skins.

The game takes place in a large, destructible underground world. The player controls a single character who mines tiles, collects items, upgrades tools, and interacts with buildings such as the Museum and Trader. Working alongside the player is an autonomous palaeontologist who digs its own tunnels, retrieves whatever it uncovers, and carries the proceeds back into the shared money balance without any input from the player.

Why create this game? To explore procedural terrain, a data-driven content pipeline, autonomous helper AI, and a satisfying progression loop inspired by mining and idle-automation games.



Main Focus

- Mining and exploration
- Collecting fossils and completing dinosaur skeletons
- Upgrading tools and improving your paleontologist helper
- Watching your autonomous helper work the dig site alongside you
- Building a sense of discovery and progression through a JSON-driven content system

What is Different

- A fully destructible world with layered hardness - all layer data defined in JSON
- A museum interior that visually updates as fossils are collected, populated entirely from JSON species data
- A data-driven fossil system using JSON configuration - new dinosaurs and

collectibles require no code changes

- A Trader upgrade system integrated with an in-game economy
- A paleontologist's helper that works the dig site independently, it chooses where to tunnel, digs its own routes, spots what it uncovers, collects it and finds its own way back to the surface

```

"map": {
  "layers": [
    {"name": "Topsoil", "texture": "ASSETS/IMAGES/Terrain/Topsoil.png", "hardness": 5},
    {"name": "Sediment", "texture": "ASSETS/IMAGES/Terrain/Sediment.png", "hardness": 30},
    {"name": "Rock", "texture": "ASSETS/IMAGES/Terrain/Rock.png", "hardness": 200},
    {"name": "Bedrock", "texture": "ASSETS/IMAGES/Terrain/Bedrock.png", "hardness": 500}
  ]
},

"museum":{
  "texture": "ASSETS/IMAGES/Sprites/Buildings/Museum.png",
  "frameWidth": 160,
  "frameHeight": 113,
  "position": {"x": 1600, "y": 338}
},

"trader":{
  "texture": "ASSETS/IMAGES/Sprites/Buildings/Trader.png",
  "frameWidth": 161,
  "frameHeight": 91,
  "position": {"x": 25, "y": 361}
},

```

```

"collectibles": [
  {
    "index": 0,
    "name": "Fossil_Fragment",
    "type": "fossil",
    "texture": "ASSETS/IMAGES/Fossils/1. Pickups/Collectibles_Sheet.png",
    "frameWidth": 64,
    "frameHeight": 64,
    "frameIndex": 0,
    "monetaryValue": 0
  },
  {
    "index": 1,
    "name": "Fossil_Fragment",
    "type": "fossil",
    "texture": "ASSETS/IMAGES/Fossils/1. Pickups/Collectibles_Sheet.png",
    "frameWidth": 64,
    "frameHeight": 64,
    "frameIndex": 1,
    "monetaryValue": 0
  },
]

```

```

"dinosaurs": [
  {
    "name": "Tyrannosaurus rex",
    "category": "Carnivore",
    "background": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Base.png",
    "skinTexture": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Skin1.png",
    "pieces": [
      { "id": "Tyrannosaurus_Skull", "texture": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Skull.png" },
      { "id": "Tyrannosaurus_Torso", "texture": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Torso.png" },
      { "id": "Tyrannosaurus_Pelvis", "texture": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Pelvis.png" },
      { "id": "Tyrannosaurus_Tail", "texture": "ASSETS/IMAGES/Fossils/Carnivore/Tyrannosaurus_Rex/Tyrannosaurus_Tail.png" }
    ]
  },
]

```

Feature Set

General Features

- Large destructible tile-based world
- Multiple terrain layers with different hardness (JSON-configured)
- Fossils, amber, and trash collectibles (JSON-defined types and values)
- Museum interior with dynamic fossil displays (populated from JSON data)
- Trader building with upgrades and hiring
- Palaeontologist helpers
- Smooth camera system with clamping
- JSON-driven configuration for all game content

Gameplay

- Mining tiles to uncover fossils
- Collecting items for money or museum completion
- Upgrading the mining ray length and damage, your jump height and pickup radius
- Upgrading the paleontologist npc mining depth so it tunnels further
- Exploring deeper layers with increasing hardness
- Viewing completed dinosaurs in the Museum Interior

The Game World

Overview

The world of PaleoPals is a vertical slice of earth, with the surface at the top and increasingly dense geological layers below. The player begins at the surface, where the Museum and Trader buildings are located, and digs downward to uncover fossils.

World Feature #1 - Destructible Terrain

The entire underground world is made of destructible tiles. Each tile has:

- A terrain type (Topsoil, Sediment, Rock, Bedrock) - loaded from JSON
- Hardness and HP values - defined in JSON
- A crack overlay that updates as the tile takes damage

World Feature #2 - Fossil Ecosystem

Fossils are distributed throughout the underground layers. When tiles break, they may spawn:

- Fossil pieces (assigned to randomly selected dinosaur species from JSON data)
- Amber (small or large - value defined in JSON)
- Trash items

These are managed by the FossilManager, which loads all dinosaur and collectible data from JSON at startup.



The Physical World

Key Locations

- Surface: Museum, Trader, starting area
- Underground: 215 rows of tiles, each row representing deeper geological layers

Scale

- Tile size: 24x24 pixels
- World width: 75 tiles
- World height: 215 tiles

Objects

Tiles, Fossils, Amber, Trash, Paleontologists, Museum interior assets, Trader menu UI



Weather / Day-Night

No weather system or day/night cycle is implemented.

Time

Time is continuous and real-time. A typical three-minute gameplay slice:

- Minute 1: Player mines surface layers, finds first fossils, visits museum to view collection
- Minute 2: Player digs deeper, encounters harder tiles, collects amber for money, while the paleontologist works on its own tunnels elsewhere on the map

- Minute 3: Player returns to surface and spends the combined earnings on tool upgrades, player upgrades or extending the npcs mining depth

Rendering System

Overview

Rendering is handled through SFML's 2D graphics API. The game uses sprites, textures, and views to draw the world efficiently.

- All assets are 2D sprites
- Tiles are drawn with frustum culling
- Collectibles are drawn only when in view
- UI is drawn in screen space

Camera

The camera follows the player smoothly and clamps to world bounds. When menus (Trader, Museum) are open, the camera switches to the default view for UI clarity.

Game Engine

Overview

The engine is a custom C++ framework built on SFML 3. It manages game states, rendering, input, physics, tile updates, upgrade application, and the NPCs decision making, which is evaluated once per frame within the same fixed step time loop as everything else

Collision is tile-based. The player checks tile hardness and HP to determine if mining is possible. Movement uses simple AABB checks. No water simulation is included.

The World Layout

Overview

The world is a vertical grid of tiles with increasing hardness. All layer properties are loaded from JSON.

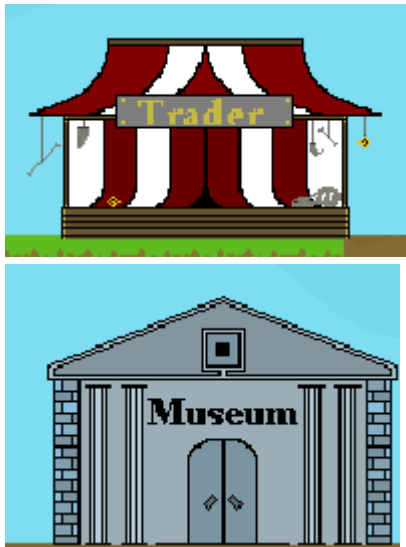
World Layout Detail #1

Topsoil -- Sediment -- Rock -- Bedrock. Each layer has unique textures and hardness values defined in config.json.

```
{
  "map": {
    "layers": [
      {"name": "Topsoil", "texture": "ASSETS/IMAGES/Terrain/Topsoil.png", "hardness": 5},
      {"name": "Sediment", "texture": "ASSETS/IMAGES/Terrain/Sediment.png", "hardness": 30},
      {"name": "Rock", "texture": "ASSETS/IMAGES/Terrain/Rock.png", "hardness": 200},
      {"name": "Bedrock", "texture": "ASSETS/IMAGES/Terrain/Bedrock.png", "hardness": 500}
    ]
  }
},
```

World Layout Detail #2

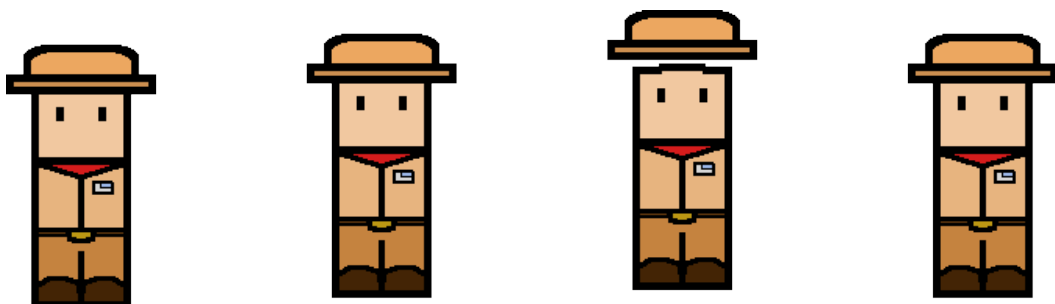
Buildings are placed on the surface using positions read from JSON. They do not move during gameplay.



Game Characters

Overview

Characters include the player and paleontologist (NPC). Character design was inspired by safari outfits, stereotypical paleontologist attire, and outfits from Jurassic Park. They both are the exact same sprite but the npc has been made smaller and green.



Paleontologist (NPC)

Behaviour:

Works without player input, cycling through its routine continuously

Patrol:

Walks the surface and has a 30% chance of starting to mine

Mine:

Once mining has started, it breaks tiles as it goes, the tunnels meander around instead of going straight downwards so no two digs look alike

Return:

When mining is finished it retraces its steps back to where it entered from

Collect:

Npc sweeps the area it has mined heading for the nearest thing it can reach and picks it up

Return Again:

Once there is nothing left worth reaching it heads back to the surface to start the process over again

Anything the paleontologist collects is done the exact same way as the player, so money is added to the same place, and fossil pieces appear in the museum too.

This is what makes upgrading it so worthwhile since the deeper it can go, the more collectibles it can find and collect without the player needing to interfere.

The npc also recovers on its own when it gets into some trouble. If no route to the surface can be found or if it spends too long trying to reach a collectible it cannot get to it will abandon the attempt and restart its routine rather than sit there idling.

Enemies and Monsters

There are no enemies in PaleoPals.

User Interface

Overview

The UI includes Money display (HUD), Trader menu, How to open the trader menu, Museum interior, How to open the museum interior, how deep the NPC search depth is, Pause menu and Main menu.

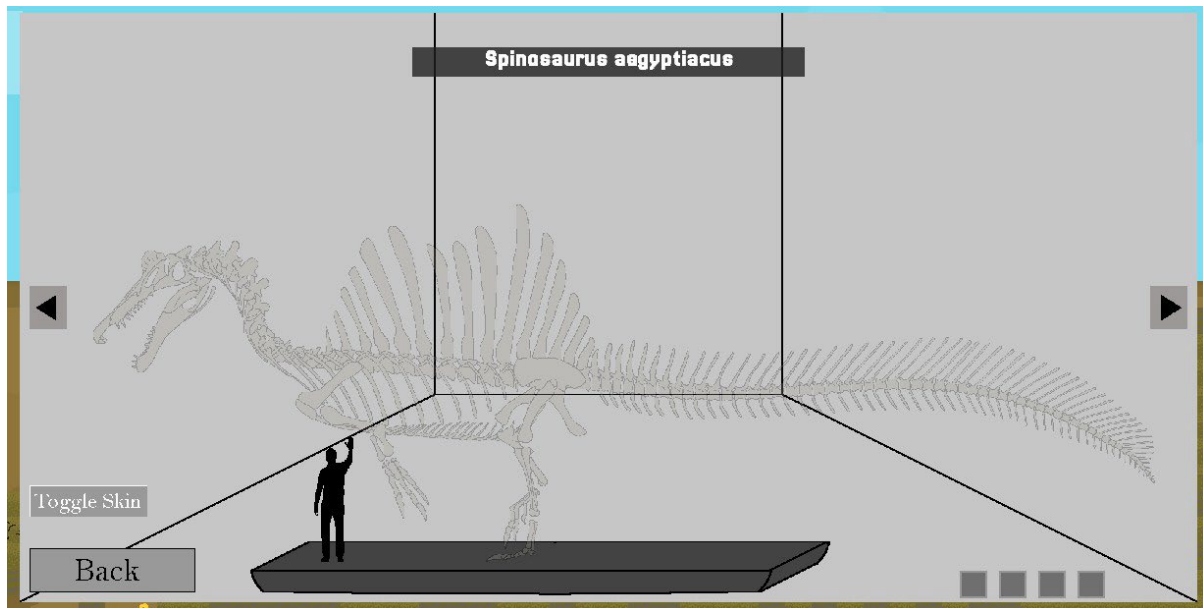
User Interface Detail #1 - Trader Menu

The Trader menu uses a tabbed layout with Upgrades and Hiring tabs. The upgrade tab holds five different purchasable upgrades, 4 of which have an affect on the player, the 5th being the npcs mining depth, each button displays the upgrade, the current level, the cost of the next level



User Interface Detail #2 - Museum Interior

The Museum Interior displays collected fossil pieces for each dinosaur species. Species data (names, piece textures, skin textures) is loaded entirely from JSON. Players can navigate between species using arrow buttons and toggle between skeleton and skin views.



Tools

The player uses a mining laser to mine tiles with the mouse. Holding left click on your mouse fires the laser from the player to the cursor, damaging tiles that cross its path. The ray has four upgradeable stats accessible through the Trader: ray length, damage, pickup radius, and jump height.

Musical Scores and Sound Effects

Sound is minimal and not a core focus of this project. Plans existed to include free sound bites for mining, finding collectibles, completing a skeleton, and digging, but these were not implemented in the final version.

Single-Player Game

Overview

PaleoPals is a single-player mining and collection game focused on progression and discovery. There is no narrative storyline; the focus is on exploration, collection, and upgrade progression.

Single Player Detail #1

The player mines tiles, collects fossils and amber, and upgrades their tool stats at the Trader. Money earned from collectibles (values defined in JSON) funds these upgrades.

Single Player Detail #2

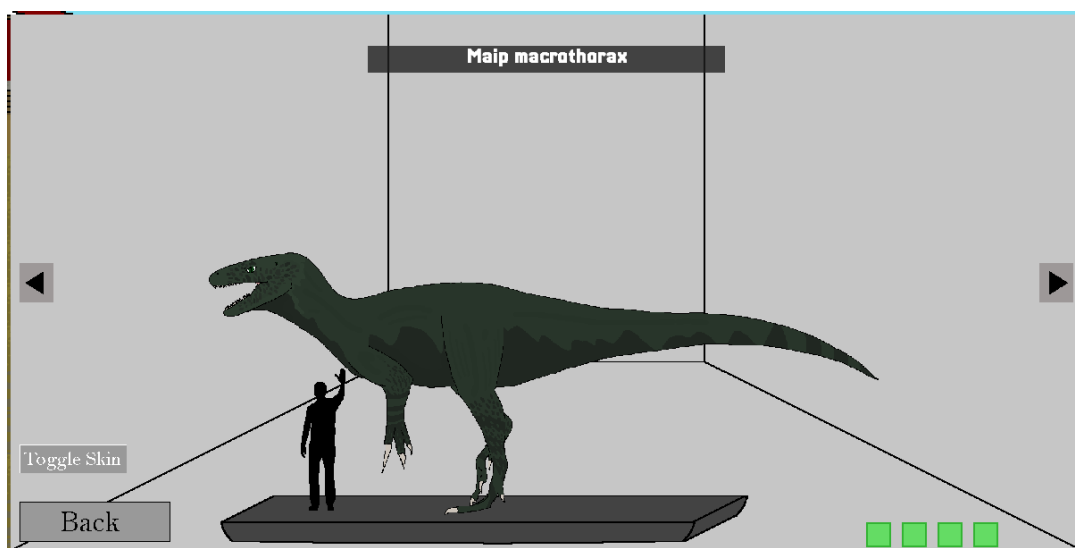
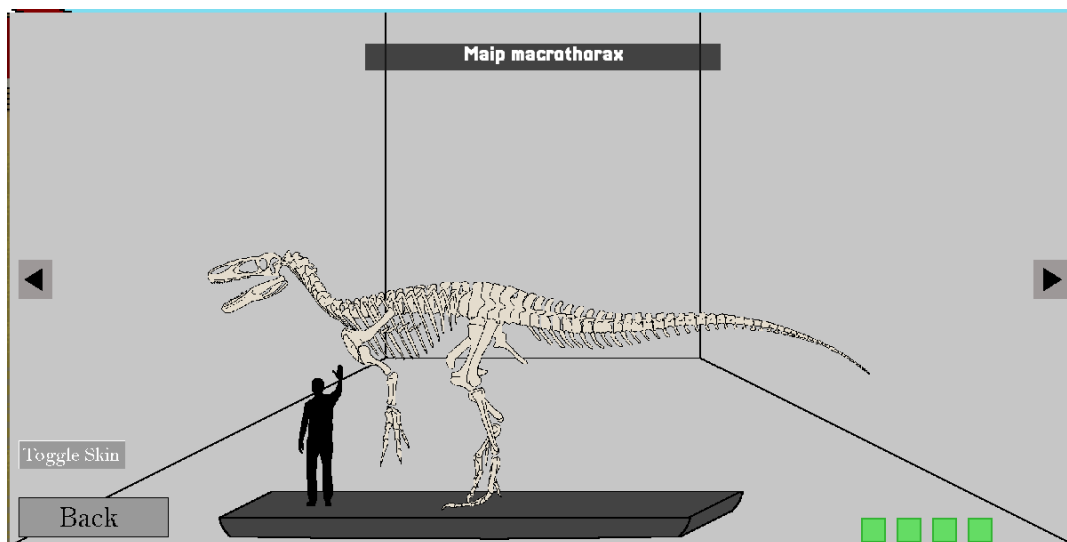
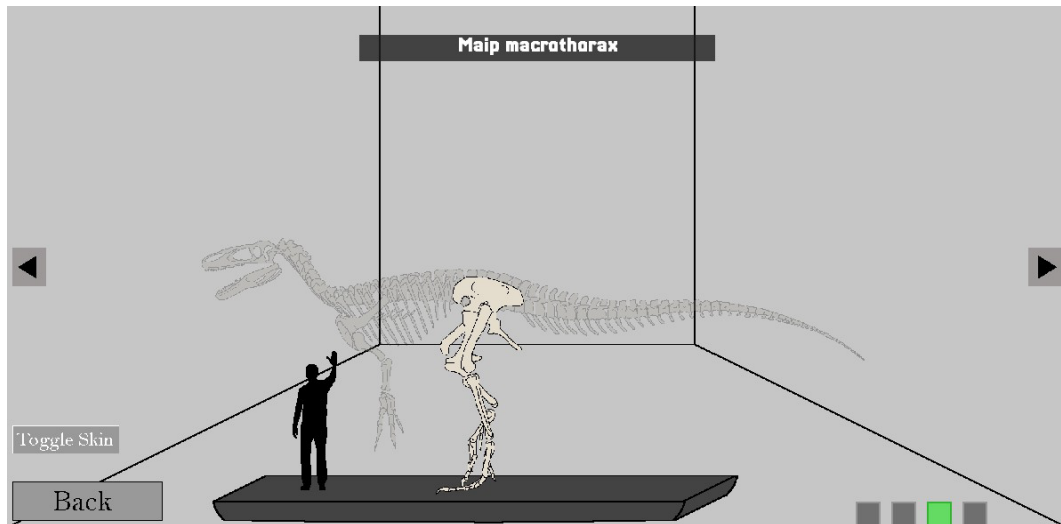
The Paleontologist(npc) works on the dig site alongside the player, mining and collecting on its own. Everything it finds goes into the same “wallet” as the player and the same museum collection.

Hours of Gameplay

Estimated 2-4 hours for full museum completion depending on fossil RNG and upgrade choices.

Victory Conditions

The game is considered complete when all dinosaur skeletons in the museum are fully assembled.



Character Rendering

Overview

Characters are rendered as animated sprites using sprite sheets with multiple frames. Animations include idle, walking, and mining states. The NPC is rendered green to differentiate it from the player.

World Editing

Overview

The world is generated from JSON configuration and is not edited in-game. Developers can modify config.json to change terrain layers, hardness values, building positions, collectible types, and dinosaur species. All tile hardness values and textures are fully data-driven.

References

Videos Used

Ögren, P. (2021). Behavior trees playlist [YouTube playlist]. YouTube.
https://www.youtube.com/playlist?list=PLFQdM4LOGDr_vYJuo8YTRemv3FrwczdKg

Computerphile. (2014, December 18). A search algorithm* [Video]. YouTube.
<https://www.youtube.com/watch?v=ySN5Wnu88nE>

Natsu Games. (2023). A pathfinding explained simply* [Video]. YouTube.
<https://www.youtube.com/watch?v=pXsPj6HewhM>

Shah, M. (2020). C++ programming playlist [YouTube playlist]. YouTube.
<https://www.youtube.com/playlist?list=PLvv0ScY6vfd8j-tlhYVPYgiIyXduu6m-L>

Books Used

Meyers, S. (1996). More effective C++: 35 new ways to improve your programs and designs. Addison-Wesley.

Documentation Used

SFML Team. (2024). SFML 3.0.0 documentation. SFML. <https://www.sfml-dev.org/documentation/3.0.0/>

Websites Used

Nlohmann. (2024). basic_json API documentation. JSON for Modern C++. https://json.nlohmann.me/api/basic_json/

Colledanchise, M. (2026). BehaviorTree.CPP source code. GitHub. <https://github.com/BehaviorTree/BehaviorTree.CPP/tree/master/src>

Colledanchise, M. (2026). Awesome Behavior Trees. GitHub. <https://github.com/BehaviorTree/awesome-behavior-trees>

Some Quick Reads that Helped:

ScholarWorks@GVSU, “Behavior Trees in Robotics and AI,” Accessed: Aug. 30, 2026. [Online]. Available: <https://scholarworks.gvsu.edu/cgi/viewcontent.cgi?article=1367&context=cistechlib>

Lisyrus, “Behavior Trees,” Accessed: Aug. 30, 2026. [Online]. Available: <https://lisyrus.github.io/blog/posts/behavior-trees.html>

Applications Used

Visual Studio Code | Visual Studio 2022 | Aseprite